

XMLTaskProcessorV2.vbs — Technical Reference

Overview

`XMLTaskProcessorV2.vbs` is a Windows login/startup script written in VBScript. It reads an XML task file (`XML_Tasks.xml`) and executes a configurable set of actions — drive mappings, registry edits, file copies, program launches, environment variable changes, and more — targeting specific users, AD groups, or computer names.

It is designed to run in two environments:

- **WTS (Windows Terminal Services / RDS)** — remote/session logon
- **PC** — standard desktop logon

Each XML task can be restricted to one or both environments using `WTS="true/false"` and `PC="true/false"` attributes.

Architecture

Startup

- └─ Load environment globals (user, computer, paths, AD groups)
- └─ Parse XML_Tasks.xml
- └─ Read <Start> element for config (order, log file, bump-off list)
- └─ Loop through RunOrder nodes
 - └─ DoSteps(xmlDoc, Context, ADGroupList, CurrentUser)
 - └─ Match elem by group / computer / user / "Global"
 - └─ Execute each child task element in order

Global Variables (set at startup)

Variable	Source	Description
<code>CURRENTUSER_GLOBAL</code>	<code>%USERNAME%</code>	Logged-in username
<code>COMPUTERNAME_GLOBAL</code>	<code>%COMPUTERNAME%</code>	Machine name
<code>USERPROFILE_GLOBAL</code>	<code>%USERPROFILE%</code>	User profile root path

Variable	Source	Description
<code>TEMPFOLDER_GLOBAL</code>	<code>%TEMP%</code>	Temp folder path
<code>DOMAINNAME_GLOBAL</code>	<code>%USERDOMAIN%</code>	AD domain name
<code>LOGONSERVER_GLOBAL</code>	<code>%LOGONSERVER%</code>	DC the user authenticated against
<code>APPDATA_GLOBAL</code>	<code>%APPDATA%</code>	Roaming AppData path
<code>LOCALAPPDATA_GLOBAL</code>	<code>%LOCALAPPDATA%</code>	Local AppData path
<code>PROGRAMFILES_GLOBAL</code>	<code>%ProgramFiles%</code>	64-bit Program Files
<code>PROGRAMW6432_GLOBAL</code>	<code>%ProgramW6432%</code>	Native 64-bit Program Files
<code>PROGRAMDATA_GLOBAL</code>	<code>%ProgramData%</code>	All-users application data
<code>PUBLICFOLDER_GLOBAL</code>	<code>%PUBLIC%</code>	Public user folder
<code>SYSTEMDRIVE_GLOBAL</code>	<code>%SystemDrive%</code>	Drive containing Windows
<code>HOMEUNC_GLOBAL</code>	<code>%HOMESHARE%</code>	UNC home share
<code>HOMEDRIVELETTER_GLOBAL</code>	<code>%HOMEDRIVE%</code>	Home drive letter
<code>HOMEPATH_GLOBAL</code>	<code>%HOMEPATH%</code>	Home path
<code>COMSPEC_GLOBAL</code>	<code>%ComSpec%</code>	Path to cmd.exe
<code>ALLUSERSPROFILE_GLOBAL</code>	<code>%ALLUSERSPROFILE%</code>	Legacy all-users profile
<code>WINDIR_GLOBAL</code>	<code>%windir%</code>	Windows directory
<code>BITNESS_GLOBAL</code>	WMI <code>Win32_Processor.AddressWidth</code>	32 or 64 — cached once at startup
<code>WTSENV</code>	Derived	<code>True</code> if running in a terminal/RDS session
<code>PCENV</code>	Derived	<code>True</code> if running on a physical PC (not WTS)
<code>LOGFILE_GLOBAL</code>	XML <code><Start LogFile="..."></code>	Path to the script's log file
<code>OSVERSIONMAJORMINOR_GLOBAL</code>	WMI <code>Win32_OperatingSystem</code>	OS version string, e.g. <code>10.0</code>

Variable	Source	Description
ADGroupList	GetADGroups()	Pipe-delimited list of user's AD groups

Path Shorthand Tokens (`Resolve` function)

Within XML attribute values, use these tokens instead of hardcoded paths:

Token	Expands to
~UP	USERPROFILE_GLOBAL
~UN	CURRENTUSER_GLOBAL
~UT	TEMPFOLDER_GLOBAL
~CN	COMPUTERNAME_GLOBAL
~AD	APPDATA_GLOBAL
~CS	COMSPEC_GLOBAL
~DN	DOMAINNAME_GLOBAL
~HS	HOMEUNC_GLOBAL
~PD	PROGRAMDATA_GLOBAL
~P6	PROGRAMFILES_GLOBAL
~P3	PROGRAMW6432_GLOBAL
~PU	PUBLICFOLDER_GLOBAL
~SY	SYSTEMDRIVE_GLOBAL
~AU	ALLUSERSPROFILE_GLOBAL
~LA	LOCALAPPDATA_GLOBAL
~HL	HOMEDRIVELETTER_GLOBAL
~HP	HOMEPATH_GLOBAL
~CD	CURRENTWORKINGFOLDER_GLOBAL
~UD	USERDISPLAYNAME_GLOBAL

Example: File="~UP\Desktop\readme.txt" → C:\Users\jdoe\Desktop\readme.txt

XML Task File Schema

<Start> — top-level configuration

```
xml

<Start
  Name="My Login Script"
  Order="Global;SomeGroup;AnotherGroup"
  BumpOff="TermedUsersGroup"
  Debug="true"
  LogFile="~HS\logs\login.log"
/>
```

Attribute	Description
Name	Descriptive name logged at startup
Order	Semicolon-delimited list of XML context names to process, in order
BumpOff	AD group name — members are immediately logged off
Debug	Enables debug output (true/false)
LogFile	Log file path (supports ~ tokens)

Context Elements — task grouping

Tasks are grouped under elements named after the context. The script matches a context element if:

- Global — always runs
- <GroupName> — runs if the user belongs to that AD group
- <UserName> — runs if the username matches exactly
- <Computer Name="PC01"> — runs if the computer name matches (supports * wildcard prefix, e.g. Name="SALES*")

Task Elements (child of a context)

<RunProgram> — run an executable

xml

```
<RunProgram CMD="notepad.exe" Arg="~UP\notes.txt"
  Wait="true" Visible="true" WTS="true" PC="true" Bits="32,64"/>
```

Attribute	Values	Description
CMD	Path or exe name	Program to run
Arg	String	Command-line arguments
Wait	true / false	Wait for process to exit before continuing
Visible	true / false	Show window (true) or run hidden (false)
WTS	true / false	Run in terminal sessions
PC	true / false	Run on physical PCs
Bits	32 , 64 , 32,64	Restrict to 32-bit or 64-bit OS

Also available: <RunProgram1> and <RunProgram2> — identical syntax, executed in sequence order within the same context.

<RunIf> — conditional program execution

xml

```
<RunIf What="FileExists" Item="~UP\marker.txt" Op="eq" Value="true"
  Do="setup.exe" DoArg="/silent"
  ElseDo="fallback.exe" ElseDoArg=""
  Wait="true" Visible="false" WTS="true" PC="true" Bits="64"/>
```

Attribute	Description
What	Name of a VBScript function to call (e.g. <code>FileExists</code> , <code>FolderExists</code> , <code>GetFileSize</code>)
Item	Argument passed to that function
Op	Comparison: <code>eq</code> , <code>neq</code> , <code>gt</code> , <code>lt</code> , <code>geq</code> , <code>leq</code> , <code>in</code> , <code>notin</code>
Value	Value to compare the function's result against
Do / DoArg	Program to run if condition is true
ElseDo / ElseDoArg	Program to run if condition is false (optional)

`<RunIfVar>` — run based on environment variable value

```
xml
<RunIfVar Var="APPENV" Op="eq" Value="PROD"
  Do="prodsetup.exe" DoArg="" ElseDo="" ElseDoArg=""
  Wait="true" Visible="false" WTS="false" PC="true" Bits="32,64"/>
```

Same as `<RunIf>` but tests an environment variable by name rather than calling a function.

`<Map>` — map drive or printer

```
xml
<!-- Drive mapping -->
<Map Drive="H:" UNC="//server\share" ForceMap="true"
  Persistent="false" WTS="true" PC="true"
  DelayBefore="0" DelayAfter="0"/>

<!-- Printer mapping -->
<Map Printer="//printserver\HP_Color" UNC="//printserver\HP_Color"
  Default="true" WTS="true" PC="false"/>
```

Attribute	Description
<code>Drive</code> / <code>Printer</code>	Drive letter (e.g. <code>H:</code>) or UNC printer path
<code>UNC</code>	Network share path
<code>ForceMap</code>	Disconnect and remap even if already mapped
<code>Persistent</code>	Survive logoff (<code>true</code> / <code>false</code>)
<code>Default</code>	Set as default printer (<code>true</code> / <code>false</code>)
<code>DelayBefore</code> / <code>DelayAfter</code>	Milliseconds to wait before/after mapping

`<Reg>` — read/write/delete registry values

xml

```
<Reg Key="HKCU\Software\MyApp" Value="Setting" Data="1"
  Type="REG_DWORD" Op="update" PC="true" WTS="true"/>
```

Attribute	Values	Description
<code>Key</code>	Registry path	Full registry key path
<code>Value</code>	String	Value name
<code>Data</code>	String	Data to write (ignored for <code>delete</code>)
<code>Type</code>	<code>REG_SZ</code> , <code>REG_DWORD</code> , <code>REG_EXPAND_SZ</code> , <code>REG_BINARY</code> , <code>REG_MULTI_SZ</code>	Registry value type
<code>Op</code>	<code>update</code> , <code>add</code> , <code>delete</code>	Operation to perform

`<SetRegIfVar>` — conditionally write a registry value

xml


```
<SetRegIfVar Var="USERDOMAIN" Op="eq" Value="CORP"
  RegKey="HKCU\Software\Corp" RegValue="Enabled"
  RegType="REG_DWORD" WTS="true" PC="true"/>
```

Writes the registry value only if the named environment variable matches the comparison.

<SetEnv> — set a user environment variable

xml

```
<SetEnv Var="MY_APP_ROOT" Value="~P6\MyApp" WTS="true" PC="true"/>
```

<AddPath> — append to the user PATH

xml

```
<AddPath Path="~P6\MyApp\bin" WTS="true" PC="true"/>
```

Appends the specified folder to the user-level **(PATH)** environment variable if not already present.

<Copy> — copy a file

xml

```
<Copy File="\\server\share\config.ini" To="~UP\AppData\MyApp\"
  OverWrite="true" WTS="true" PC="true"/>
```

<CopyFolder> — copy a folder tree

xml

```
<CopyFolder Folder="\\server\share\templates" To="~UP\Documents"
  WTS="true" PC="true"/>
```

<Create> — create a folder, file, or shortcut

```
xml

<!-- Folder -->
<Create Folder=~UP\MyApp\Logs" WTS="true" PC="true"/>

<!-- Empty file -->
<Create File=~UP\MyApp\marker.txt" OverWrite="false" WTS="true" PC="true"/>

<!-- Shortcut -->
<Create Shortcut="My App" Location=~UP\Desktop\My App.lnk"
  Cmd=~P6\MyApp\app.exe" WorkDir=~P6\MyApp"
  Icon=~P6\MyApp\app.exe,0" HotKey="" WindowStyle="1"
  WTS="true" PC="true"/>
```

<ShowProgress> / <EndProgress> — progress display

```
xml

<ShowProgress Text="Configuring your environment..." Banner="IT Setup"
  Type="1" WTS="true" PC="true"/>
<!-- ... tasks ... -->
<EndProgress WTS="true" PC="true"/>
```

Launches an HTA progress indicator (`ShowProgress1.hta` etc.) and terminates it. The `Type` attribute selects the HTA variant.

<PopupMessage> / <PopupMessageFirst> / <PopupFinishMessage> — user dialogs

```
xml

<PopupMessage Text="Setup is complete. Please restart."
  Caption="IT Notice" TimeOut="30" WTS="true" PC="true"/>
```

`PopupMessageFirst` runs before most other tasks; `PopupFinishMessage` runs at the end. `TimeOut` in seconds (0 = no timeout).

<ClearProcess> — prompt user to close running applications

xml

```
<ClearProcess
  ProcList="outlook.exe|Microsoft Outlook,teams.exe|Microsoft Teams"
  Caption="Please close applications" ElseMsg="Click Cancel to abort."
  ForProcName="MyApp Setup" WTS="true" PC="true"/>
```

`ProcList` is a comma-delimited list of `process.exe|Friendly Name` pairs. The user is shown a dialog listing any that are running and asked to close them before continuing. Cancelling exits the script with code `777`.

<Sleep> — pause execution

xml

```
<Sleep TimeOut="3000"/>
```

Pauses for the given number of milliseconds.

Utility Functions Reference

File / Folder

Function	Returns	Description
<code>FileExists(path)</code>	Boolean	True if file exists
<code>FolderExists(path)</code>	Boolean	True if folder exists
<code>GetFileSize(path)</code>	Long	File/folder size in bytes, or -1 if missing
<code>FileCreated(path)</code>	Date	File creation date
<code>FileLastModified(path)</code>	Date	Last modified date
<code>FileLastAccessed(path)</code>	Date	Last accessed date
<code>FileAge(path)</code>	Double	Age in days since last modification

Function	Returns	Description
LinesInFile(path)	Integer	Total line count
fRead(path)	String	Full file contents (ANSI or Unicode)
Head(path, n)	String	First <code>n</code> lines
Tail(path, n)	String	Last <code>n</code> lines (single-pass circular buffer)
VerFile(path)	String	File version number
IsUnicodeFile(path)	Boolean	True if file starts with Unicode BOM
IsHiddenFile(path)	Boolean	True if hidden attribute is set
isFileAttribute(path, attr)	Boolean	Test <code>hidden</code> , <code>system</code> , <code>readonly</code> , <code>archive</code> , <code>link</code> , <code>compressed</code>
ShowFileAccessInfo(path)	String	Formatted file metadata summary

Folder

Function	Returns	Description
FolderCreated(path)	Date	Folder creation date
FolderSize(path)	Long	Total folder size in bytes
FolderSet(path)	Array	All subfolders recursively
LargestNumberedSubFolder(path)	Integer	Highest-numbered direct subfolder
folIsEmpty(path)	String	<code>"True"</code> if folder has no files
HasRights(path)	String	<code>"True"</code> if user has list rights
IsHiddenFolder(path)	Boolean	True if hidden attribute is set
isFolderAttribute(path, attr)	Boolean	Test <code>hidden</code> , <code>system</code> , <code>readonly</code> , <code>folder</code> , <code>archive</code> , <code>compressed</code>
DeleteIfEmpty(path)	—	Deletes folder if it contains nothing

File I/O

Sub/Function	Description
<code>fWrite(path, data)</code>	Overwrite file with data
<code>fAppend(path, data)</code>	Append data to file
<code>WriteFile(path, data, type)</code>	Write with explicit encoding (<code>ASCII</code> or <code>UNICODEFILE</code>)
<code>AppendFile(path, data, type)</code>	Append with explicit encoding
<code>ReadFile(path, type)</code>	Read with explicit encoding
<code>NewFile(path, overwrite)</code>	Create empty file
<code>DeleteFile(path)</code>	Delete file if it exists
<code>LogItToFile(path, msg)</code>	Append timestamped log entry
<code>TrimFileSize(path, size)</code>	Trim oldest lines until file is under <code>size</code> bytes

Path Helpers

Function	Returns	Description
<code>BaseName(path)</code>	String	Filename portion of a full path
<code>PathName(path)</code>	String	Directory portion of a full path
<code>FileExt(path)</code>	String	Extension (without dot)
<code>GetWinDir()</code>	String	Windows directory
<code>GetWinSys()</code>	String	Windows\System32 directory
<code>GetWinTemp()</code>	String	Windows temp directory
<code>GetTempName()</code>	String	Random temp filename
<code>GetTempFullPath()</code>	String	Full path to a random temp file
<code>TempFileName()</code>	String	Alias for <code>GetTempName</code>
<code>TempFileBase()</code>	String	Full path using a random 8-char name

Drives & Disks

Function/Sub	Description
<code>GetDiskFree(drive)</code>	KB free on a drive
<code>ShowDriveType(drive)</code>	Human-readable drive type string
<code>CdRomDrives()</code>	Array of CD-ROM drive letters
<code>GetRemovableDrive()</code>	First removable drive path
<code>FreeDrive(first, last)</code>	First available drive letter in range
<code>ValidDriveLetter(drive)</code>	True if letter is D-Z
<code>Map(drive, unc, persist)</code>	Map a network drive
<code>ForceDriveReMap(drive, unc, persist)</code>	Disconnect then remap
<code>NTFSCompressFolder(path)</code>	NTFS-compress a folder
<code>NTFSCompressFile(path)</code>	NTFS-compress a single file

Registry

Function/Sub	Description
<code>RegExists(key)</code>	True if registry key/value exists
<code>GetRegVal(key)</code>	Read a registry value
<code>UpdateReg(key, value, data, type)</code>	Write a registry value
<code>DeleteReg(item)</code>	Delete a registry key or value
<code>ExportReg(key, file)</code>	Export registry branch to <code>.reg</code> file
<code>ImportReg(file)</code>	Import a <code>.reg</code> file

Process / Execution

Function/Sub	Description
<code>RunProgram(cmd, wait)</code>	Run a program in a visible window
<code>RunProgramHidden(cmd, wait)</code>	Run a program hidden
<code>ExecProgram(cmd)</code>	Run and return process ID
<code>ExecProgramCaptureOutput(cmd)</code>	Run and return stdout + exit code
<code>ExecProgramCaptureDelimitedOutput(cmd, delim, type)</code>	Run and return delimited output
<code>KillProcess(name)</code>	Terminate all processes matching name
<code>KillSession()</code>	Force logoff current session
<code>QuitNow(code)</code>	Exit script with given code

Environment

Function/Sub	Description
<code>GetEnv(var)</code>	Expand an environment variable
<code>SaveLocalVar(name, value)</code>	Set a user-level environment variable
<code>GetLocalEnv(name)</code>	Read a user-level environment variable
<code>AddPathLocal(folder)</code>	Append folder to user PATH

OS / System

Function	Description
<code>Is64BitOS()</code>	True if running on 64-bit OS (cached)
<code>Is32BitOS()</code>	True if running on 32-bit OS (cached)
<code>FindOSVersionMajorMinor()</code>	Returns e.g. <code>"10.0"</code>

Function	Description
<code>FindOSVersionFull()</code>	Returns full version string
<code>IsRunningAsAdministrator()</code>	True if running elevated
<code>IsDcUp(server)</code>	True if domain controller is reachable
<code>IsShare(path)</code>	True if UNC/disk path is accessible
<code>GetADGroups(domain, user)</code>	Pipe-delimited AD group list

Printing

Sub	Description
<code>MapPrinter(unc, default)</code>	Connect to a network printer
<code>SetDefPrinter(unc)</code>	Set default printer
<code>DeletePrinter(unc)</code>	Remove printer connection
<code>DetectLocalPrinter(name)</code>	True if named local printer exists

Miscellaneous

Function/Sub	Description
<code>TF(x)</code>	Returns <code>True</code> for <code>"true"</code> , <code>"1"</code> , <code>"yes"</code> , <code>"on"</code>
<code>GetVal(x)</code>	Returns <code>1000</code> for yes/on/1, <code>0</code> for no/off/0, else <code>CInt(x)</code>
<code>Resolve(item)</code>	Expands <code>~XX</code> path tokens
<code>Evalf(func, item, op, val)</code>	Evaluates <code>func(item) op val</code>
<code>EnvMatch(wts, pc)</code>	True if current environment matches WTS/PC flags
<code>Delay(ms)</code>	Sleep for <code>ms</code> milliseconds
<code>Chop(str)</code>	Remove last character
<code>ChopChar(str, char)</code>	Remove last character if it matches <code>char</code>

Function/Sub	Description
<code>ChopChop(str, before, after)</code>	Trim before/after delimiters
<code>RandomNumb(hi, lo)</code>	Random integer between <code>lo</code> and <code>hi</code>
<code>ElementsInArray(arr)</code>	Total element count
<code>NonEmptyElementsInArray(arr)</code>	Count of non-empty elements
<code>IsDST(date, arr)</code>	Returns 1 if date is in DST, 0 if not
<code>LowerCaseFSO(path)</code>	Lowercase all filenames in a folder tree
<code>XCopy(from, to)</code>	xcopy with <code>/S /Y</code> flags
<code>CopyFolder(src, dest)</code>	Copy folder if destination doesn't exist
<code>Copy(from, to, overwrite)</code>	Copy file(s)
<code>MoveFile(old, new)</code>	Move/rename a file
<code>md(path)</code>	Recursively create directory path
<code>HideFolder(path) / UnHideFolder(path)</code>	Set/clear hidden attribute on folder
<code>HideFile(path) / UnHideFile(path)</code>	Set/clear hidden attribute on file
<code>SetReadOnlyFile(path) / UnReadOnlyFile(path)</code>	Set/clear read-only attribute
<code>IncludeVbsScript(path)</code>	Execute another <code>.vbs</code> file in global context
<code>FreeDrive(first, last)</code>	First unused drive letter in range
<code>CreateShortCut(...)</code>	Create a Windows shortcut (.lnk)
<code>RemoveFoldersOverNDays(path, n)</code>	Delete folders/files older than n days
<code>LogEvent(type, msg)</code>	Write to Windows Event Log

Execution Flow Summary

1. **Global initialization** — environment variables, AD groups, OS bitness cached.

2. **XML load** — `XML_Tasks.xml` is parsed. Parse errors show a message box.
 3. **<Start> config read** — script name, run order, log file, terminated-user group.
 4. **Log directory created** — `md()` ensures the log path exists.
 5. **BumpOff check** — if user is in `TermedUsers` AD group, `KillSession()` is called.
 6. **Task loop** — for each node in `RunOrder`, `DoSteps()` is called, which:
 - Iterates all matching context elements (Global, group, user, computer)
 - Executes every child task element in document order
 - Applies `WTS`/`PC` and `Bits` filters to each task
 - Resolves `~XX` tokens in all path values before use
-

Key Design Notes

- **All actions are logged** to `LOGFILE_GLOBAL` via `LogItToFile()`.
- `Resolve()` is called on all path-bearing attributes before use — always use `~` tokens in XML rather than hardcoded paths.
- `TF()` normalizes boolean XML attributes — `"yes"`, `"1"`, `"on"`, and `"true"` all mean `True`.
- `BITNESS_GLOBAL` is populated once at startup via WMI; `Is32BitOS()` / `Is64BitOS()` are cheap reads thereafter.
- `Tail()` uses a circular buffer — single file pass regardless of line count.
- `FolderSet()` recurses correctly — a prior bug (double-incrementing loop index) has been fixed.
- `LogItToFile()` auto-creates the log file if absent — no pre-creation step needed.